NORTH STAR SPRINT
GROUP PROJECT

1. **SUMMARY;** Northstar Retail Co. asked for a working prototype that lets customers self-resolve common support questions without a human agent in the loop. The Team Charter set the bar at deflecting at least two tickets: order status, returns & refunds, and stock availability .The delivered prototype is a Streamlit-based self-serve dashboard (app.py) backed by two CSV data sources (orders.csv, stock.csv). It meets the charter's bar and goes one further: all three target ticket types are implemented and working, plus a fourth self-serve surface — an FAQ page — that catches the remaining long tail of repeat questions. The app is published as a public GitHub repository with a README covering installation and usage, and is also live at the linked hosted URL for stakeholder demo access.
2. **PROBLEM STATEMENT**

The Northstar has a problem with repetitive, low-complexity questions — "where is my order," "when do I get my refund," "is this back in stock." Each one currently requires an agent to look up a record a customer could look up themselves, given the right interface. That drives up response time for genuinely complex tickets and keeps cost-per-contact high for issues that don't need a human judgment call.
We are building a tool that is able to help without a lot of human interference and answer most of these questions directly from existing order and inventory data, deflecting tickets before they're opened. This is going to be a replacement for the queue on this subset of intents.

3. **APPROACH TO RESOLVING THE PROBLEM**

3.1 Ticket categories covered

- Order Status — customer enters an Order ID and receives item, quantity, order date, current status, and a plain-language next step (e.g. "your order is on its way").
- Returns & Refunds — customer enters an Order ID and receives return status, refund amount, return reason, and a status-specific next step (requested / in transit / received / completed).

- Stock Availability — customer searches by product ID or name and receives live quantity, a low-stock warning below the reorder threshold, or a restock date if the item is out of stock.
- FAQ — eight pre-written answers to the most common surrounding questions (tracking, non-delivery, return timelines, order changes), so a query that doesn't fit the three structured lookups still gets deflected.

3.2 Design principle

Every screen follows the same three-step shape: ask for one identifier (Order ID, or product name/ID) → look it up against the data → show status plus a concrete next action. That consistency is what keeps a 5-page app usable without an onboarding flow — a customer who learns the Order Status page already knows how Returns & Refunds works.

3.3 Tech stack

A) Frontend / app framework; Python — chosen for familiarity and speed: a working, styled multi-page UI in one file, no separate frontend build

B) Data; Pandas over two flat CSV files (orders.csv, stock.csv) — no database needed for MVP scope; @st.cache_data avoids re-reading on every interaction

C) Hosting ; prototype at northstar-support-buddy.lovable.app for stakeholder access without a local install

D) Source control; Public GitHub repository (MIT-licensed code, proprietary data files) with README covering setup, usage, and file structure

3.4 Data model

orders.csv holds one row per order: order_id, item, quantity, order_date, order_status, return_status, refund_amount, return_reason. stock.csv holds one row per product: product_id, product_name, quantity_in_stock, reorder_threshold, restock_date. Both are read once per session and cached, keeping every lookup a simple in-memory filter rather than a live query.

4. RESULTS

4.1 What works end-to-end

- Order Status lookup returns item, quantity, order date, and status for a valid Order ID, with color-coded status (delivered / shipped / processing) and a matching next-step message.
- Returns & Refunds lookup returns return status, refund amount, and return reason for a valid Order ID, correctly distinguishing "no return requested" from an in-progress or completed return.
- Stock Check resolves by exact product ID or partial, case-insensitive product name, and correctly branches into in-stock, low-stock (≤ reorder threshold), and out-of-stock (with restock date) states.
- FAQ page surfaces eight common questions in an expandable format requiring no data lookup.
- Error handling: an unmatched Order ID or product query returns a clear "not found" message with a hint, instead of a blank result or a stack trace.

4.2 Known limitations

- Data is static CSV, not a live order-management or inventory system — a production version needs a real data integration.
- No authentication: any Order ID can be looked up by anyone who has it, which is acceptable for an MVP demo but not for production (should be paired with the account/email on the order).
- No analytics on which questions are being deflected vs. escalated, so deflection rate can't yet be measured quantitatively from the app itself.
- Single-language, text-only interface — no voice or multilingual support.

5. TEAM PROCESS AND WORK DISTRIBUTION

Delivery followed the Team Charter's structure: a 10-task board (each task capped at 4 hours per the anti-black-box rule), same-day status updates, and a conflict-resolution ladder (direct → pod vote → instructor escalation) that was never needed to be invoked past step one.

The board balanced two tasks per seat across all five seats, with high-priority items (charter, board setup, repo, core implementation) front-loaded into the first half of the sprint.

In this report; only the aggregate delivery outcome is produced.

6. NEXT STEPS

- Replace static CSVs with a live connection to Northstar's order management and inventory systems.
- Add lightweight identity check (email or account match) before returning order/return details.
- Instrument the app to log lookup volume and "not found" rate as a proxy for deflection impact.
- Expand FAQ content based on real post-launch question logs rather than the initial assumed set.